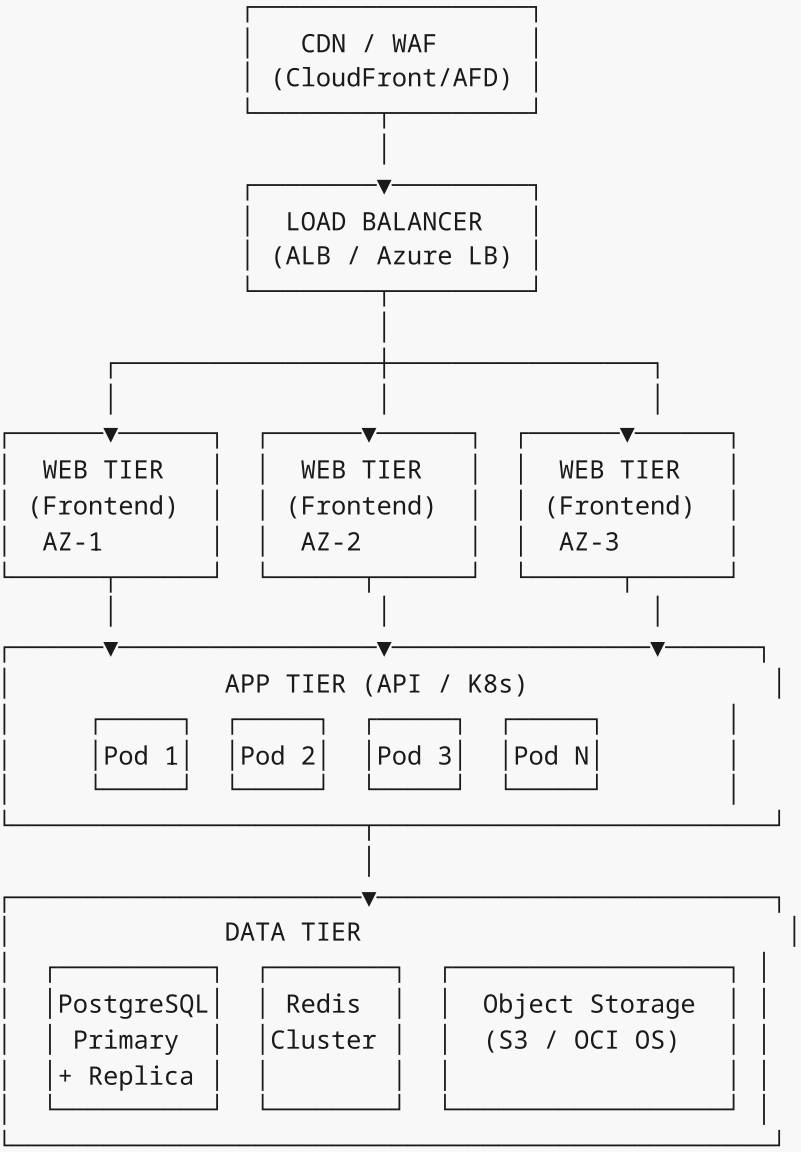
Part 8 — Architecture, HA, DR & Scalability

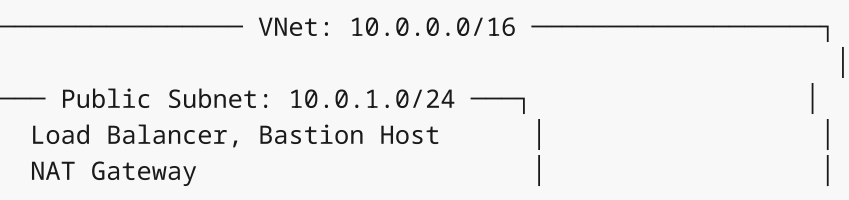
Chapter 21: Cloud Architecture

Probability: 🟡 65%

Multi-Tier Architecture



VPC/VNet Design



```
│   └──────────────────────────────────────┘                    │
│                                                               │
│   ┌─── Private Subnet (App): 10.0.10.0/24 ──┐                 │
│   │  Application Servers / K8s Nodes        │                 │
│   │  No public IPs — egress via NAT Gateway │                 │
│   └─────────────────────────────────────────┘                 │
│                                                               │
│   ┌─── Private Subnet (Data): 10.0.20.0/24 ──┐                │
│   │  Databases, Redis, Internal Services     │                │
│   │  No internet access                      │                │
│   └──────────────────────────────────────────┘                │
│                                                               │
│  NSG Rules:                                                   │
│  • Public: Allow 80/443 from internet                         │
│  • App: Allow from LB only                                    │
│  • Data: Allow from App subnet only                           │
└───────────────────────────────────────────────────────────────┘
```

Chapter 22: High Availability

Probability: 🟡 65%

HA Patterns

Pattern	Implementation	Your Experience
Multi-AZ	Deploy across availability zones	Kubernetes nodes across AZs
Database HA	Primary-replica with auto-failover	PostgreSQL + Patroni, Redis Sentinel
Load Balancing	Distribute traffic	Nginx, HAProxy, cloud LBs
Health Checks	Detect failures	K8s probes, LB health checks
Auto-healing	Replace failed instances	K8s restarts crashed pods
Stateless design	No server-side sessions	Sessions in Redis, state in DB

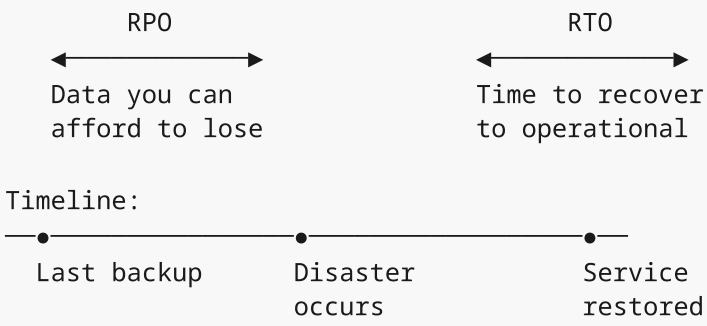
Q: "How do you achieve 99.9% uptime?"

> *"99.9% = 8.77 hours downtime per year. We achieve this through: 1) Multi-AZ deployment — no single point of failure. 2) Database HA with Patroni — automatic failover in seconds. 3) Kubernetes self-healing — crashed pods restart automatically. 4) Rolling updates with zero-downtime strategy. 5) Proactive monitoring with SLO-based alerts. 6) Documented runbooks for rapid incident response."*

Chapter 23: Disaster Recovery

Probability: 🟡 60%

RTO vs RPO



Tier	RPO	RTO	Strategy	Cost
Tier 1	~0	< 1 hour	Active-Active, synchronous replication	$$$$
Tier 2	< 1 hour	< 4 hours	Warm standby, async replication	$$$
Tier 3	< 24 hours	< 24 hours	Pilot light, scheduled backups	$$
Tier 4	> 24 hours	> 24 hours	Backup & restore only	$

Q: "Describe your DR strategy"

> *"We implement Tier 2 DR: automated snapshots every 6 hours with cross-region replication. PostgreSQL WAL archiving provides point-in-time recovery. Kubernetes manifests are in Git — we can recreate the entire application layer in a new region using Terraform + kubectl. Our tested RTO is sub-30 minutes. We run DR drills quarterly to validate."*

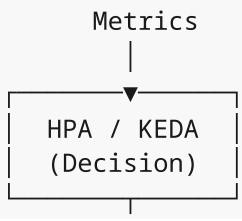
Chapter 24: Scalability

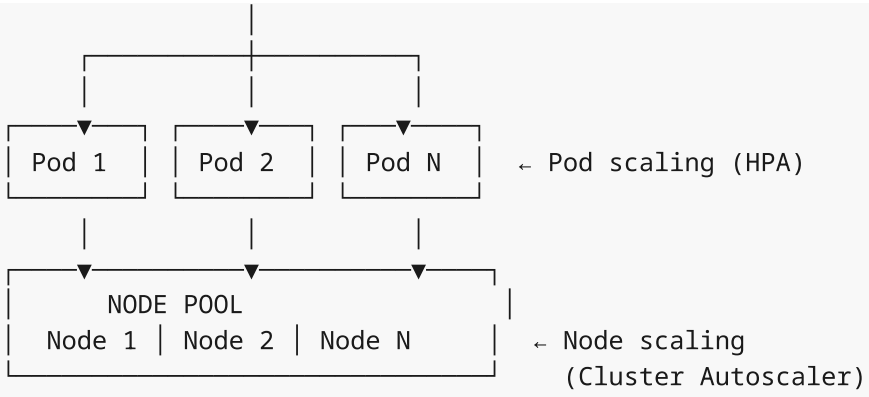
Probability: 🟡 55%

Horizontal vs Vertical Scaling

Type	How	Pros	Cons
Vertical	Bigger machine	Simple, no code changes	Single point of failure, hardware limits
Horizontal	More machines	Infinite scale, fault tolerant	Needs stateless design, complexity

Auto-Scaling Architecture





50 Architecture Interview Questions

#	Question	Key Points
1	Design a CI/CD pipeline for microservices	Monorepo vs polyrepo, parallel builds, artifact promotion, environment isolation
2	How would you migrate a monolith to microservices?	Strangler fig pattern, start with non-critical services, API gateway
3	Design a highly available web application	Multi-AZ, LB, auto-scaling, database replication, CDN
4	How do you handle database migrations in CI/CD?	Backward-compatible changes, separate migration step, rollback plan
5	Design a multi-environment infrastructure	Separate VNets, shared modules, environment-specific configs
6	How do you handle secrets across environments?	Key Vault per environment, separate service principals, RBAC
7	Design a monitoring strategy for 100+ microservices	Prometheus federation, Grafana dashboards per team, alert routing
8	How do you implement blue-green deployment?	Two identical environments, DNS/LB switch, instant rollback
9	How do you implement canary deployment?	Gradual traffic shift (1% → 5% → 25% → 100%), automated rollback on error rate
10	Design a disaster recovery plan	Multi-region, backup replication, automated failover, quarterly testing
11	How do you optimize cloud costs?	Rightsizing, reserved instances, spot/preemptible, idle cleanup, tagging
12	Design a logging architecture at scale	Fluent Bit → Loki/Elasticsearch, retention policies, structured logging
13	How do you handle certificate management?	cert-manager + Let's Encrypt for external, internal CA for mTLS

#	Question	Key Points
14	Design a GitOps workflow	Git as source of truth, ArgoCD/Flux, automated sync, drift detection
15	How do you secure a Kubernetes cluster?	RBAC, NetworkPolicy, PodSecurity, image scanning, audit logging
16	How do you handle noisy neighbor in multi-tenant K8s?	ResourceQuotas, LimitRanges, separate namespaces, priority classes
17	Design infrastructure for a startup going from 100 to 10,000 users	Start simple (managed services), plan for horizontal scaling, cache layer
18	How do you handle database schema versioning?	Flyway/Liquibase, backward compatible, CI-integrated
19	How do you implement zero-downtime deployments?	Rolling updates, readiness probes, connection draining, pre-stop hooks
20	Design a service mesh architecture	Istio/Linkerd for mTLS, traffic management, observability — but add complexity
21	How do you handle config across 50+ microservices?	Centralized config (ConfigMaps, Consul), environment-specific overrides
22	How do you test infrastructure changes safely?	Terraform plan review, staging environment, canary infrastructure
23	Design a backup strategy	3-2-1 rule (3 copies, 2 media, 1 offsite), automated, tested restores
24	How do you manage multiple Kubernetes clusters?	Fleet management (Rancher/ArgoCD), consistent policies, centralized monitoring
25	How do you implement RBAC at organization scale?	Groups/teams in AD/LDAP, mapped to K8s roles, audit regularly
26	Design an incident response system	PagerDuty/Opsgenie, runbooks, escalation matrix, blameless postmortems
27	How do you handle stateful applications in K8s?	StatefulSets, persistent volumes, operator pattern (e.g., PostgreSQL operator)
28	How do you implement IaC governance?	Sentinel/OPA policies, PR reviews, automated `tflint` / `checkov` in CI
29	Design a multi-cloud strategy	Abstract with Terraform modules, avoid cloud-specific lock-in where possible
30	How do you handle legacy application containerization?	Assess dependencies, create Dockerfile, test extensively, run in compatibility mode
31	Design API gateway architecture	Rate limiting, auth, routing, versioning, circuit breaker
32	How do you implement feature flags?	LaunchDarkly/Unleash, decouple deploy from release, gradual rollout

#	Question	Key Points
33	How do you handle data sovereignty/compliance?	Region-specific deployments, data residency controls, encryption, audit trails
34	Design a self-service platform for developers	Internal developer portal, golden path templates, automated environments
35	How do you measure DevOps maturity?	DORA metrics: lead time, deployment frequency, MTTR, change failure rate
36	How do you handle technical debt in infrastructure?	Allocate sprint capacity (20%), track in backlog, prioritize by risk
37	Design auto-scaling for unpredictable traffic	HPA + cluster autoscaler, pre-warming, KEDA for event-driven
38	How do you handle cross-region latency?	CDN, regional deployments, database read replicas per region
39	How do you secure CI/CD pipelines?	Signed commits, pipeline-as-code, secret masking, RBAC, audit logs
40	Design a platform engineering team's responsibilities	Golden paths, IDP, self-service infra, SRE practices, developer experience
41	How do you handle vendor lock-in?	Use open standards (K8s, Terraform), abstract cloud-specific APIs
42	Design observability for serverless	Distributed tracing, structured logging, custom metrics, X-Ray/Jaeger
43	How do you handle ephemeral environments?	Terraform workspaces or dynamic namespaces, auto-cleanup, PR-based
44	How do you implement chaos engineering?	Start with game days, Litmus/Chaos Monkey, controlled experiments, blast radius limits
45	How do you handle container image supply chain security?	Base image scanning, signed images, private registry, SBOM
46	Design a FinOps practice	Tagging strategy, cost dashboards, budget alerts, optimization reviews
47	How do you handle secrets rotation?	Automated rotation (Key Vault), short-lived tokens, workload identity
48	Design an SRE on-call rotation	Follow-the-sun, primary+secondary, runbooks, toil budget, escalation
49	How do you handle compliance audits?	Automated compliance checks, audit logs, policy-as-code, documentation
50	What's your approach to infrastructure documentation?	ADRs (Architecture Decision Records), runbooks, diagrams-as-code, living docs